

Chunking Techniques

30 September 2026 14:14

Choosing a chunking strategy in RAG is mainly about answering this question:

"What amount of information should I retrieve together so that the LLM gets enough context to answer, but not so much irrelevant context?"

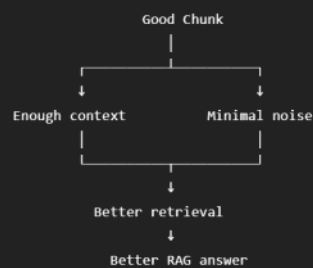
There is **no single best chunk size**. It depends on your document type, query type, embedding model, and retrieval method.

1. First understand the trade-off

Too small
↓
Context is incomplete
↓
LLM may miss important information

Too large
↓
Too much irrelevant information
↓
Retrieval becomes less precise
↓
More tokens + higher latency

The goal is:



2. How I choose the strategy

I normally evaluate these 5 factors:

A. Document structure

Ask:

Does the document have meaningful sections?

For example, an aerospace maintenance manual:

Chapter 1 - Engine Overview
Chapter 2 - Engine Components
Chapter 3 - Temperature Monitoring
Chapter 4 - Fault Detection
Chapter 5 - Maintenance Procedures

Here, **structure-aware/recursive chunking** makes sense.

Don't blindly split every 500 tokens.

B. Query type

Consider what users will ask.

Short factual questions

"What is the normal engine temperature?"

Smaller chunks can work well.

Procedure questions

"What are the steps to troubleshoot engine overheating?"

You need the **whole procedure**, so chunks should preserve procedural context.

Complex analytical questions

"What are the possible causes of abnormal engine temperature and how should each one be diagnosed?"

You may need larger chunks or retrieve multiple related chunks.

3. Main chunking strategies

Strategy 1 — Fixed-size chunking

Example:

Chunk size = 500 tokens
Overlap = 50 tokens

Document

Chunk 1: 1 — 500

Chunk 2: 450 — 950

Chunk 3: 900 — 1400

Use when

- Documents have poor structure
- Plain TXT files
- Logs
- Large raw text
- You need a simple baseline

Advantage

Very easy to implement.

Disadvantage

Can break a meaningful sentence, paragraph, or procedure.



4. Strategy 2 — Recursive chunking

This is often a good starting point for general-purpose RAG.

Instead of immediately cutting at 500 tokens, try natural boundaries:

```
Document
↓
Paragraph
↓
Sentence
↓
Smaller boundary
↓
Character/token limit
```

For example:

```
Chapter
↓
Section
↓
Paragraph
↓
Sentence
```

You might configure:

```
chunk_size = 500
chunk_overlap = 50
```

but let the splitter try to preserve natural boundaries.

Good for

- Manuals
- Reports
- Technical documents
- Policies
- General PDFs
- Knowledge bases

5. Strategy 3 — Semantic chunking

Here you split based on **meaning**, rather than simply token count.

Imagine:

```
Paragraph 1 → Engine temperature monitoring
Paragraph 2 → Temperature sensors
Paragraph 3 → Sensor calibration
```

```
Paragraph 4 → Radar configuration
Paragraph 5 → Radar signal processing
```

A semantic splitter may detect that:

```
P1 + P2 + P3
```

belong together, while:

```
P4 + P5
```

belong to another topic.

Use when

- Topics change frequently
- Paragraphs have different semantic meanings
- Retrieval quality is more important than ingestion simplicity

Disadvantage

More computationally expensive and requires tuning.



6. Strategy 4 — Structure-aware chunking

For your defence/aerospace RAG, this is particularly useful.

Suppose you have:

```
Aircraft Maintenance Manual
```

```
1. Engine System
  1.1 Engine Overview
  1.2 Temperature Monitoring
  1.3 Fault Detection

2. Hydraulic System
  2.1 Hydraulic Components
  2.2 Pressure Monitoring
```

Instead of:

```
Chunk 1
Chunk 2
Chunk 3
```

create chunks with their hierarchy:

```
{
  "text": "...",
  "metadata": {
    "document": "Aircraft Maintenance Manual",
    "chapter": "Engine System",
    "section": "Temperature Monitoring"
  }
}
```

This gives your retriever more useful information.

7. Strategy 5 — Document-specific chunking

This is very important.

Don't use the same chunking strategy for every document type.

For example:

Document	Suitable approach
TXT	Recursive/fixed
Technical manual	Structure-aware
SOP	Procedure/step-aware
Legal document	Section/clause-aware
Tables	Table-aware
Logs	Event/time-window based
Code	Function/class/module-aware
FAQs	One Q&A pair per chunk
Research paper	Section-aware
Reports	Heading + paragraph-aware

8. Special case: procedures

Suppose your document says:

Engine Overheating Procedure

Step 1: Reduce engine power.

Step 2: Check temperature sensor.

Step 3: Inspect cooling system.

Step 4: Record temperature readings.

Step 5: Contact maintenance control.

Don't blindly create:

Chunk 1:

Step 1

Step 2

Chunk 2:

Step 3

Step 4

Chunk 3:

Step 5

You may lose the relationship between the steps.

Better:

Chunk:

Engine Overheating Procedure

Step 1...

Step 2...

Step 3...

Step 4...

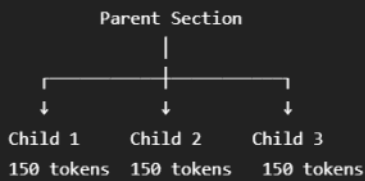
Step 5...

Or use parent-child retrieval.



9. Parent-child chunking

This is useful when you need both **precise retrieval** and **larger context**.



Search using the small child chunks.

But when one child matches:

```
graph TD; A[Child 2 matches query] --> B[Retrieve Parent Section]; B --> C[Give larger context to LLM];
```

The flowchart shows the process when a child chunk matches a query. It starts with 'Child 2 matches query', which leads to 'Retrieve Parent Section', which then leads to 'Give larger context to LLM'.

This can work well for technical manuals.

Technique	How it Works	Pros	Cons	When to Use
Fixed-size chunking	Split every N tokens (e.g., 500)	Very fast	Breaks context	Small/simple documents
Sliding window	500 tokens with 100-token overlap	Preserves context	Duplicate data	Most production RAG systems
Sentence-based	Split at sentence boundaries	Better readability	Uneven chunk sizes	Articles, reports
Paragraph-based	One paragraph per chunk	Preserves meaning	Some chunks may be too long	PDFs with clear formatting
Semantic chunking	Split when topic changes	High retrieval quality	Computationally expensive	High-quality enterprise search
Hierarchical chunking	Document → Section → Paragraph	Excellent context	More complex	Large manuals, books

Fixed-size Chunking:

- **Straight forward** approach to chunking.
- Simply decide the no.of tokens in our chunk and use this number to break up our documents into fixed size chunks.
- **Max context window size** of the embedding model (such as **1024** for **llama-text-embed-v2**, **8196** for **text-embedding-3-small**).

Notes: Fixed-sized chunking will be the best path in most cases, and we recommend starting here and iterating only after determining it insufficient.

Simple Sentence and Paragraph splitting

- **Naive splitting:** This approach would be to split sentences by periods ("."), new lines, or white space.

- **NLTK**: It provides a trained sentence tokenizer that can split the text into sentences, helping to create more meaningful chunks.
- **spaCy**: It offers a sophisticated sentence segmentation feature that can efficiently divide the text into separate sentence

Recursive Character Level Chunking:

LangChain implements a [RecursiveCharacterTextSplitter](#) that tries to split text using separators in a given order. The default behavior of the splitter uses the ["\n\n", "\n", " ", ""] separators to break paragraphs, sentences and words depending on a given chunk size.

Document structure-based chunking

When chunking large documents such as PDFs, DOCX, HTML, code snippets, Markdown files and LaTeX, specialized chunking methods can help preserve the original structure of the content during chunk creation.

- **PDF** documents contain loads of headers, text, tables, and other bits and pieces that require preprocessing to chunk.
- **HTML**, from scraped web pages can contain tags (<p> for paragraphs, or <title> for titles) that can inform text to be broken up or identified, like on product pages or blog posts. Roll your own parser.
- **Markdown** is a lightweight markup language commonly used for formatting text. By recognizing the Markdown syntax (e.g., headings, lists, and code blocks), you can intelligently divide the content based on its structure and hierarchy, resulting in more semantically coherent chunks.
- **LaTeX** is a document preparation system and markup language often used for academic papers and technical documents. By parsing the LaTeX commands and environments, you can create chunks that respect the logical organization of the content (e.g., sections, subsections, and equations), leading to more accurate and contextually relevant results.

Semantic Chunking

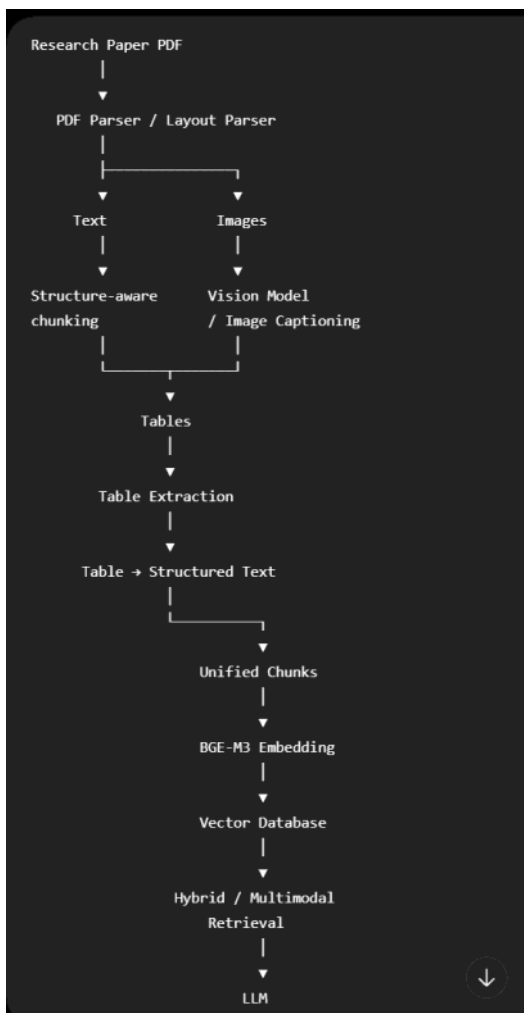
A new experimental technique for approaching chunking was first introduced by [Greg Kamradt](#)

if you're building an application with LLMs, you most likely already have the ability to create embeddings - and embeddings can be used to extract the semantic meaning present in your data. This semantic analysis can be used to create chunks that are made up of sentences that talk about the same theme or topic.

Contextual Chunking with LLMs

Sometimes, it's not possible to chunk information from a larger complex document without losing the context entirely. This can happen when the documents are many hundreds of pages, change topics frequently, or require understanding from many related portions of the document. Anthropic introduced [contextual retrieval in 2024](#) to help address this problem.

For a research-paper RAG system containing text + images + tables, I would not use one chunking technique for everything. Use a multimodal, structure-aware pipeline.



1. Text → Structure-aware + semantic/recursive chunking

Research papers have meaningful structure:

```

Title
Abstract
1. Introduction
2. Related Work
3. Methodology
  3.1 Dataset
  3.2 Proposed Method
4. Experiments
5. Results
6. Conclusion
References
  
```

Don't simply split every 500 tokens.

Instead:

```

Paper
↓
Section
↓
Subsection
↓
Paragraphs
↓
Recursive/semantic chunks
  
```

Example metadata:

```

Python
{
  "paper_id": "P001",
  "section": "3.2 Proposed Method",
  "page": 5,
  "content_type": "text",
  "chunk_id": "P001-S3.2-C04"
}
  
```

For a first implementation, 400–700 tokens with ~10% overlap is a reasonable starting experiment.

2. Tables → Table-aware extraction

This is important.

Suppose the paper contains:

Model	Accuracy	F1	mAP
ResNet	91.2	89.5	84.2
ViT	94.1	92.8	88.7

Don't treat the table as an image only.

Extract it into a structured representation:

Table: Model Performance

Model: ResNet
Accuracy: 91.2
F1: 89.5
mAP: 84.2

Model: ViT
Accuracy: 94.1
F1: 92.8
mAP: 88.7

Store:

```
Python
{
  "content_type": "table",
  "table_id": "T03",
  "page": 7,
  "section": "4. Experiments",
  "caption": "Model Performance"
}
```

This makes questions such as:

"Which model achieved the highest mAP?"

much easier to retrieve and answer.

For complex tables, preserve the **original table structure** as well as a text representation.

3. Images / figures → Vision-based processing

Research papers often contain:

Figure 1: Proposed Architecture
Figure 2: Model Pipeline
Figure 3: Experimental Results
Figure 4: Attention Visualization

Simply extracting:

[IMAGE]

loses valuable information.

Use a **vision-language model (VLM)** or image-captioning/OCR pipeline to generate a description.

For example:

Figure 2: Proposed Architecture

The figure shows an input image entering a CNN backbone.
The extracted features are passed to a transformer encoder,
followed by a classification head.

Store:

```
Python
{
  "content_type": "image",
  "image_id": "F02",
  "page": 4,
  "section": "3. Methodology",
  "caption": "Proposed Architecture",
  "description": "The figure shows..."
}
```

For scientific plots, also preserve **OCR/text and chart-specific information** rather than relying only on a generic caption.

4. Equations → Equation-aware extraction

Research papers also have mathematical equations.

For example:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

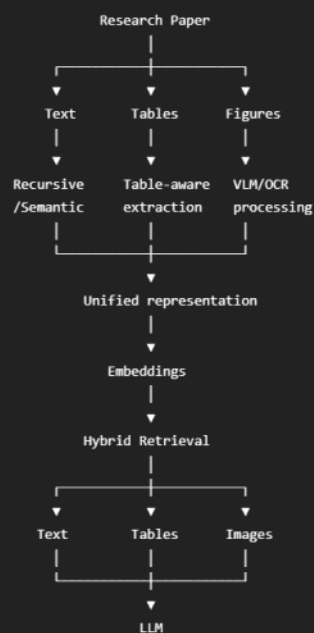
You should preserve the equation as a distinct content object:

```
Python
{
  "content_type": "equation",
  "equation_id": "EQ085",
  "page": 6,
  "section": "3.2",
  "text": "Attention(Q,K,V) = ..."
}
```

Don't let a chunking operation arbitrarily split an equation from the explanation immediately surrounding it.

5. The key technique: multimodal/structure-aware chunks

For research papers, I would use something like:



6. What I would use for your RAG project

Content	Technique	Representation
Normal text	Structure-aware + recursive	Text chunks
Sections	Hierarchical metadata	Section → subsection
Tables	Table-aware extraction	Structured text/JSON + original table
Images	VLM + OCR	Caption/description + image
Figures	Figure-aware	Caption + visual description
Charts	VLM/chart extraction	Description + values when recoverable
Equations	Equation-aware	LaTeX/text
References	Separate/low-priority chunks	Citation metadata